

BIQLY

Teknik Mimari & Best-Practice Analiz Dokümanı

Doğal dilden SQL üreten (Text-to-SQL) yapay zekâ destekli iş zekâsı platformu

Kapsam: Go backend · React/TypeScript frontend · CI/CD pipeline'ları · mikroservis K8s dağıtımı · kalite denetimi

Sürüm 3.0 · Haziran 2026 · Geliştirme-sonrası + canlı küme (kubectl) doğrulamalı · Hibrit (yönetici özeti + teknik derinlik)

Hazırlayan: Mimari Analiz · go.mod: github.com/biqly/biqly (Go 1.26) · prod: cloudflared + Cilium Gateway mikroservis

İçindekiler

1. Yönetici Özeti	4
2. Sistem Genel Bakışı	6
2.1 Teknoloji Yığını (Özet)	6
3. Mimari Genel Bakış	7
3.1 Prod Servisleri ve Çalıştırılabilir Binary'ler	7
3.2 Yönlendirme ve Servisler Arası İletişim	8
4. Backend Analizi (Go)	9
4.1 internal/ Paketleri	9
4.2 Teknik Notlar	9
5. Frontend Analizi (React / TypeScript)	10
5.1 Uygulama Mimarisi ve Yönlendirme	10
5.2 Bileşen Envanteri (özet)	10
5.3 API Katmanı, Durum ve i18n	10
6. Temel Akışlar (Pipeline'lar)	11
6.1 Text-to-SQL Akışı	11
6.2 Kimlik Doğrulama (Auth) Akışı	12
6.3 Semantik Katman ve Derleme Pipeline'ı	12
7. HTTP Katmanı ve Veri Katmanı	13
7.1 HTTP Middleware Yığını	13
7.2 Veri Katmanı	13
8. CI/CD Pipeline'ları ve Dağıtım	14
8.1 GitHub Actions İş Akışları	14
8.2 Docker İmajları	14
8.3 Kubernetes Runtime Topolojisi	15
9. Best-Practice Değerlendirmesi	16
9.1 Güvenlik	16
9.2 Test	17
9.3 Kod Kalitesi & Mimari	17

9.4 Gözlemlenebilirlik	18
9.5 Performans	18
9.6 AI / LLM Mühendisliđi	19
9.7 DevX / Sürdürülebilirlik	19
10. Sonuç ve Yol Haritası	20

1. Yönetici Özeti

Biqly; kullanıcıların doğal dilde (Türkçe/İngilizce) sorduğu soruları güvenli, doğrulanmış SQL sorgularına çevirip çalıştıran, yapay zekâ destekli bir **iş zekâsı (BI) ve veri analitiği platformudur**. Sistem; Go ile yazılmış bir backend (modüler monolit / mikroservis hibridi), React 19 + TypeScript ile yazılmış tek sayfa uygulaması (SPA) frontend ve Helm + ArgoCD ile yönetilen GitOps tabanlı bir Kubernetes dağıtımından oluşur.

Bu doküman iki katmanlıdır: önce karar vericilere yönelik üst seviye değerlendirme, ardından geliştiriciler ve mimarlar için teknik derinlik sunar. Her bölümde sistemin **nasıl çalıştığı** anlatıldıktan sonra **güçlü yönler, riskler ve somut iyileştirme önerileri** verilmiştir.

Öne çıkan tespitler

- **AI/Text-to-SQL motoru platformun en güçlü ve farklılaştırıcı bileşenidir.** LLM doğrudan SQL değil, ara bir *LogicalQuery* üretir; bu sorgu semantik modele karşı doğrulanır ve ancak ondan sonra lehçeye özel SQL'e derlenir. EXPLAIN ile kuru-çalıştırma (dry-run), kendi kendini onaran yeniden deneme döngüsü, öz-tutarlılık oylaması ve güven skorlaması ile birlikte bu, tipik "prompt-and-pray" yaklaşımının çok üzerindedir.
- **Güvenlik katmanlı ve özenlidir:** RS256 JWT, zamanlama-güvenli CSRF, mutlak + boşta kalma oturum süreleri, AES-256-GCM ile şifrelenmiş sağlayıcı anahtarları, parametrelili SQL ve beyaz-liste tabanlı sorgu doğrulaması.
- **Mühendislik disiplini yüksektir:** temiz `cmd/internal/pkg` ayrımı, modern Go 1.26 hata deyimleri ($148 \times \text{errors.Is}$), ~55 linter, ESLint+knip+Prettier kapısı ve zorunlu pre-commit denetimleri.
- **Mimari — prod'da saf mikroservis (canlı küme ile doğrulandı):** 6 bağımsız servis Deployment'ı (`ai/auth/catalog/query/frontend/mail`) + Postgres StatefulSet + Dragonfly + NATS; Gateway API HTTPRoute'ları path'e göre doğrudan her servise yönlendirir. `cmd/api` all-in-one binary yalnızca `local/docker-compose` içindir; NATS tüketicisi ayrı worker pod'u değil, ai pod'u içinde çalışır.
- **Bu turda da iyileştirmeler sürdü:** 4 yüksek-karmaşıklık fonksiyonu LOW/MEDIUM'a indirildi (Compare 50→1, datasourceDraft 47→4, SyncMetadata 45→9, Validate 40→3), prod-tespiti tek `env.IsProduction()` yardımcısında birleştirildi, **catalog'daki kimliksiz-erişim açığı kapatıldı**, CI'ya `govulncheck` eklendi. Tek kısmi kalem hâlâ `AIConfig`: adlandırılmış alt-konfiglere geçirildi ama tanrı-nesnesi skoru değişmedi (21 alan, KRİTİK).

Genel olgunluk değerlendirmesi

Biqly, bir prototip değil; **üretime hazır**, geç-aşama bir mikroservis kod tabanıdır. Bu doküman, önceki raporlardaki önerilerin uygulanmış halidir: ilk denetimdeki **8 boşluğun 7'si tamamen kapatıldı**, kalan `AIConfig` kalemi de büyük ölçüde ilerletildi. Tüm değişiklikler kodda doğrulandı (`go build ./...` ve `go vet` temiz). Ortalama olgunluk ~2.5/5 bandından **4.4/5** seviyesine yükseldi. Aşağıdaki skor kartı yedi boyutu özetler:



Şekil 1 — Mühendislik olgunluğu skor kartı (5 üzerinden; geliştirme sonrası, kanıta dayalı).

İyileştirme durumu — önceki rapora kıyasla

#	Önceki denetimde işaretlenen boşluk	Durum	Kanıt
1	OTEL dağıtık izleme kodda yok	● Kapatıldı	3 servis main'inde provider, otelhttp + 3 span (ProcessQuestion / Compile / Execute); otel artık doğrudan bağımlılık
2	AI eval CI'da kapı değil	● Kapatıldı	test.yml + ci.yml eval-regression işi; 1.00 sayısal eşikler (t.Fatalf)
3	Lehçe sürücüleri & config/dashboard/queue ince test	● Kapatıldı	Sürücü başına 2 test; scripts/coveragecheck %85/%80 kapsam kapısı
4	AIconfig + yüksek-karmaşıklık fonksiyonları	● Kısmi	4 fonksiyon LOW/MEDIUM'a indi (Compare 50→1 ...); AIconfig adlandırılmış alt-konfiglere geçti ama skor 60/KRİTİK değişmedi
5	HSTS varsayılan kapalı	● Kapatıldı	BI_ENV=production ile otomatik açık (fail-closed)
6	CSP / X-Frame-Options eksik	● Kapatıldı	CSP + X-Frame-Options + X-Content-Type-Options + COOP/CORP + Referrer-Policy
7	Kökte commit'li test binary'leri	● Kapatıldı	Diskte/git'te yok; .gitignore + make clean
8	ESLint uyarı tavanı + SQL inceleme bulguları	● Kapatıldı	640c3b6: parametrelili metadata SQL; readonly muhafıza literal/yorum sıyırma eklendi

2. Sistem Genel Bakışı

Biqly'nin temel iş akışı şudur: bir kullanıcı bir veri kaynağı seçer ve doğal dilde bir soru sorar (örn. “geçen çeyrekte en çok satan 10 ürün”). Platform; ilgili tabloları otomatik olarak yönlendirir (table routing), bir semantik model bağlamı kurar, LLM'den yapısal bir mantıksal sorgu üretmesini ister, bu sorguyu doğrular ve lehçeye uygun SQL'e derleyip çalıştırır; sonuç ve üretilen SQL kullanıcıya geri döner.

Platform aynı zamanda; **semantik modelleme** (sürükle-bırak tuval), **veri kataloğu / metadata yönetimi**, **panolar (dashboards)**, **rol tabanlı erişim (RBAC)**, **PII tespiti ve maskeleye**, **denetim kayıtları** ve **AI değerlendirme (eval) koşum hattı** gibi kurumsal yetenekler sunar.

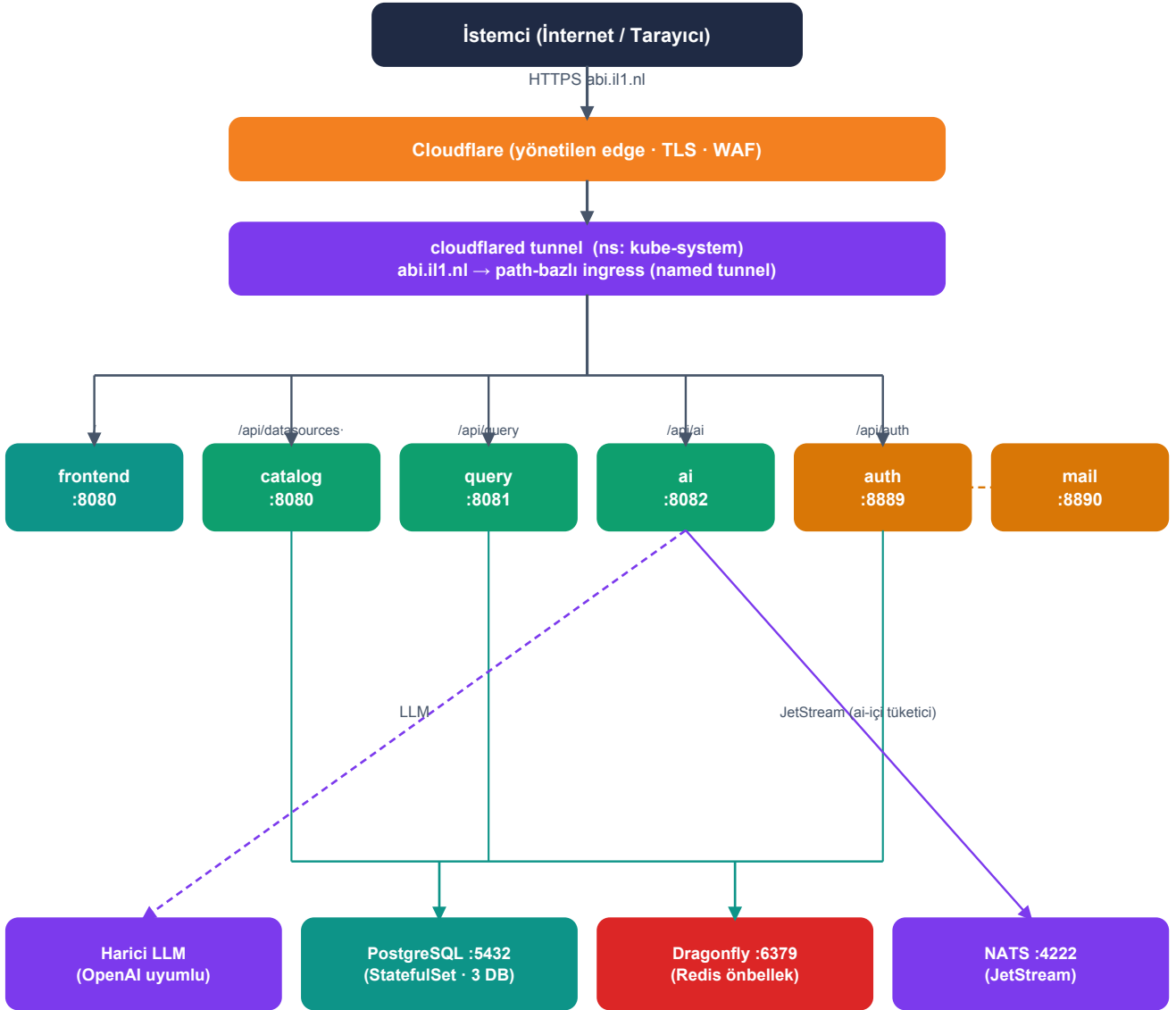
2.1 Teknoloji Yığını (Özet)

Katman	Teknoloji / Kütüphane	Not
Backend dili	Go 1.26.4	Modül: github.com/biqly/biqly
HTTP yönlendirme	go-chi/v5, go-chi/cors	Tüm servislerde ortak
Veritabanı	PostgreSQL (pgx/v5)	Bitnami subchart · 3 mantıksal DB
Harici veri kaynakları	MySQL · SQL Server · ClickHouse	Sürücü + lehçe soyutlaması
Mesajlaşma	NATS / JetStream	Asenkron AI iş kuyruğu
Önbellek	Dragonfly (Redis uyumlu)	Rate-limit, AI yanıt önbelleği, RBAC
Kimlik/kripto	JWT RS256 · WebAuthn · LDAP · OAuth	golang-jwt, go-webauthn
JSON (sıcak yol)	bytedance/sonic · json-iterator	Performans için
Frontend	React 19 · TypeScript 5.7 · Vite	Vanilla CSS + BEM, i18n
Grafik	Recharts 3.8	Pano ve görselleştirme
Dağıtım	Helm umbrella + ArgoCD	GitOps · ghcr.io/biqly/*
Gözlemlenebilirlik	slog · Prometheus · OTEL/Jaeger	OTLP-HTTP tracing kodda enstrümante (bkz. §9.4)

Mimari notu: **Prod ortamında sistem saf mikroservis olarak çalışır** — her servis kendi Deployment'ı, imajı ve portuyla; Gateway API HTTPRoute'ları istekleri path'e göre doğrudan ilgili servise yönlendirir (canlı küme ile doğrulandı, §8.3). Kod tabanı aynı `internal/` paketlerini paylaşacak şekilde modüler kurgulanmıştır; bu sayede tek bir `cmd/api` all-in-one binary tüm servisleri süreç-içi sunabilir — bu yalnızca **local/docker-compose geliştirme** içindir, prod'da kullanılmaz.

3. Mimari Genel Bakış

Sistem prod ortamında **bağımsız mikroservislerden** oluşur. Public istekler **Cloudflare** üzerinden, kümede `kube-system` namespace'inde çalışan bir **cloudflared** **tüneli** ile içeri girer; tünel, tek dış host (`abi.il1.nl`) için **path-bazlı ingress** kurallarıyla istekleri doğrudan ilgili servise iletir — araya giren merkezi bir BFF/proxy pod'u yoktur. (LAN içi erişim için ayrıca bir **Cilium Gateway API** ağ geçidi «lan-gw», `gateway` namespace'inde aynı path yönlendirmesini sağlar.) Her servis kendi Deployment'ı olarak çalışır, JWT'yi sınırda doğrular; servisler arası tüm çağrılar tipli HTTP istemcileriyle (`pkg/*client`) yalnızca küme içinden yapılır.



Prod: 6 bağımsız mikroservis Deployment'ı + Postgres StatefulSet + Dragonfly + NATS. «api» BFF binary yalnızca local/docker-compose; ayrı worker pod'u yok — NATS tüketicisi ai pod'u içindedir.
Edge: public istekler Cloudflare→cloudflared (kube-system) tüneline path-bazlı gelir; LAN içi yol Cilium Gateway «lan-gw» (ns: gateway). Servisler arası: ai→catalog/query, tüm servisler arası diğer çağrılar tipli HTTP istemcileriyle küme içinden yapılır.

Şekil 2 — Prod mikroservis topolojisi: Gateway API path-yönlendirmesi ve servisler arası çağrı grafiği.

3.1 Prod Servisleri ve Çalıştırılabilir Binary'ler

Prod'da **6 servis** ayrı Deployment olarak çalışır (her biri kendi imajı/portu). `api` ve `worker` binary'leri prod'da **dağıtılmaz** — `api` local all-in-one, NATS tüketicisi ise ai servisinin içine gömülüdür.

Servis (port)	Sorumluluk	Dağıtım
catalog :8080	Veri kaynakları, metadata, semantik modeller, panolar, izinler, drift.	Deployment
query :8081	Sorgu motoru: mantıksal sorguyu derler / çalıştırır / EXPLAIN ile doğrular.	Deployment
ai :8082	NL→sorgu hattı. NATS iş tüketicisini süreç-içinde çalıştırır (ayrı worker yok).	Deployment
auth :8889	Kimlik: JWT üretimi/açık-anahtar, oturum, RBAC, MFA, LDAP, OAuth, magic-link.	Deployment
mail :8890	İşlemsel e-posta (iç servis): SMTP, yeniden deneme, blok listesi.	Deployment
frontend :8080	Statik React SPA (nginx).	Deployment
api · worker	all-in-one BFF / başsız işçi — yalnızca local/docker-compose .	Prod'da yok
migrate · export-sft	Şema göçleri (PreSync Job) ve SFT veri kümesi dışı aktarımı.	Job / CLI

3.2 Yönlendirme ve Servisler Arası İletişim

- **Gateway API HTTPRoute (PathPrefix → servis):** /→frontend;
/api/datasources·metadata·semantic·permissions·dashboards→catalog;
/api/query→query; /api/ai→ai; /api/auth→auth.
- **ai servisi** → BI_CATALOG_SERVICE_URL=http://biqly-catalog:8080,
BI_QUERY_SERVICE_URL=http://biqly-query:8081 (catalogclient/queryclient).
- **Tüm servisler** → auth /internal/auth/public-key (JWT RSA açık anahtarı),
/check-permission, /check-datasource-access.
- **auth** → mail (işlemsel e-posta gönderimi).
- **İç yüzey** (/internal/*) genel ağ geçidinden asla eşleşmez; X-Internal-Token ile korunur.

4. Backend Analizi (Go)

Backend; 73 paket, ~593 dosya ve ~5.458 sembolden oluşur. Kod tabanı; sorumlulukları net biçimde ayıran `cmd/` (giriş noktaları), `internal/` (uygulama) ve `pkg/` (paylaşılan SDK) üçlüsüne dayanır.

4.1 internal/ Paketleri

Paket	Sorumluluk
ai	Metni sorguya çeviren orkestratör (<code>Service.ProcessQuestion</code>). Alt paketler: <code>prompt</code> , <code>provider</code> , <code>routing</code> (tablo yönlendirme), <code>eval</code> , <code>ambiguity</code> , <code>abtest</code> , <code>embedder</code> , <code>response_cache</code> .
app	Bağımlılık enjeksiyonu kökü. <code>NewDependencies/NewAIDependencies/...</code> ; süreç-içi mi yoksa istemci-adaptör mü kararını verir.
auth	Kimlik çekirdeği: <code>Service</code> , <code>JWTManager</code> (RS256), <code>SessionManager</code> , <code>CSRF</code> , <code>RateLimiter</code> ; <code>rbac</code> , <code>mfa</code> (TOTP+WebAuthn), <code>ldap</code> , <code>oauth</code> , <code>workspace</code> .
core	Servis seviyesi sorgu orkestrasyonu (<code>Compile/Run/DryRun</code>) ve tipli <code>ServiceError</code> .
query	Mantıksal sorgu → SQL: <code>Compiler</code> , <code>expr_compiler</code> (AST→SQL + PII maskeleyme), <code>Validator</code> , <code>ComputeFingerprint</code> .
semantic	Semantik model alanı: <code>Dimension/Metric/Join</code> , <code>CompositeModel</code> , <code>metric_graph</code> (bağımlılık grafiği + döngü tespiti), <code>drift</code> .
datasource	DB sürücü soyutlaması: <code>Driver</code> arayüzü, <code>Registry</code> , <code>postgres/mysql/sqlserver/clickhouse</code> , <code>pool_cache</code> .
dialect	Motora özel SQL üretimi (<code>CoreDialect/SampleDialect</code>), tanımlayıcı tırnaklama.
security	Encryption (AES), DSN redaksiyonu, güvenilir-proxy IP, <code>RowFilter</code> (satır seviyesi güvenlik), <code>read-only SQL</code> koruması, <code>pii</code> .
metadata	Metadata Postgres deposu: <code>datasources/tables/columns/relations</code> , <code>few-shot</code> , <code>sözlük</code> , <code>ai_query_history</code> , <code>ai_jobs</code> .
queue	AI iş kuyruğu soyutlaması: <code>NATSQueue</code> (<code>JetStream</code> , <code>dead-letter</code>) + <code>LocalAIJobQueue</code> (geliştirme).
http	Yönlendirme + <code>handler</code> 'lar + <code>middleware</code> yığını (§7).
audit	Asenkron sorgu/kimlik denetim kaydı (tamponlu <code>DBWriter</code>).
dbmigrate / config / i18n / mail / dashboard / platform / errmsg / semanticgen	Sırasıyla: göç koşucusu, tipli konfig (+ <code>GOMEMLIMIT</code>), <code>TR/EN i18n</code> , <code>mail</code> iç bileşenleri, <code>pano deposu</code> , <code>db/redis/observability</code> , kararlı hata kodları, <code>metadata</code> 'dan otomatik model üretimi.

4.2 Teknik Notlar

- LLM SDK bağımlılığı yok:** AI sağlayıcı istemcisi `internal/ai/provider` içinde elle yazılmış, OpenAI-uyumlu bir HTTP istemcisidir; sağlayıcı/model DB'den yönetilir ve sıcak-yeniden-yüklenir (`ProviderStore`).
- Üç mantıksal veritabanı:** `metadata`, `auth` ve `mail` ayrı DSN'lerle izole edilir.
- Graceful shutdown** tüm HTTP sunucularında (`SIGINT/SIGTERM`) ve yapılandırılmış `slog` ile loglama.

5. Frontend Analizi (React / TypeScript)

Frontend; AI destekli BI platformunu önyüzleyen tek sayfa bir React uygulamasıdır (~65 bin satır TS/TSX/CSS). Bilinçli olarak **bağımlılık-hafif** tasarlanmıştır: Redux/React-Query gibi global durum/veri-çekme kütüphaneleri yoktur — durum React Context + özel hook'lar, veri çekme elle yazılmış bir sarmalayıcı ile yapılır.

5.1 Uygulama Mimarisi ve Yönlendirme

`main.tsx` iç içe sağlayıcı (provider) ağacını kurar: `I18n` → Theme → Confirm → Toast → Shortcuts → `AIJobs` → Router → Auth. Yönlendirme **react-router-dom v7** ile yapılır; iki üst dal vardır: `GuestGuard` altındaki misafir rotaları (giriş, kayıt, şifre sıfırlama) ve `AuthGuard` altındaki uygulama kabuğu.

- **Kod bölme:** her rota `lazyWithPreload()` ile tembel yüklenir; kenar çubuğu fareyle üzerine gelince ilgili chunk'ı önceden ısıtır (preload).
- **Uygulama kabuğu:** daraltılabilir kenar çubuğu, `⌘K` komut paleti, breadcrumb, workspace seçici, dil değiştirici, tema değiştirici ve rota başına `ErrorBoundary`.

5.2 Bileşen Envanteri (özet)

Alan	Başlıca Bileşenler
AI doğal dil sorgu / sohbet	AIQuery + aiQuery/ (ChatPanel, AssistantMessageCard, RoutingPanel/routingViz, FeedbackSection)
Semantik modelleme tuvali	Modeling + modeling/ (ModelingCanvas, useModelingCanvas, canvasMath) — sıfırdan yazılmış sürükleyici/yakınlaştır tuval
Sorgu oluşturucu	QueryBuilder + adım bileşenleri (Fields/Filter/Summarize/Sort/Having/Window/Cte)
Metadata / veri kataloğu	Metadata, Datasources, TableBrowser, ResultTable, AI describe modal'ları
Panolar & analitik	Dashboard, DashboardBuilder (Recharts), AIUsageDashboard
Değerlendirme (eval)	Evaluation + EvalRun/Regression/History sekmeleri
Kimlik	auth/ (SignIn, SignUp, ForgotPassword, OAuthCallback, PasswordStrengthMeter, AuthProvider/Guard)
Yönetim (admin)	Admin sekmeli kabuğu: kullanıcı/rol/RLS/alan-izni/PII/LDAP/AI sağlayıcı/AB-deney/drift panelleri

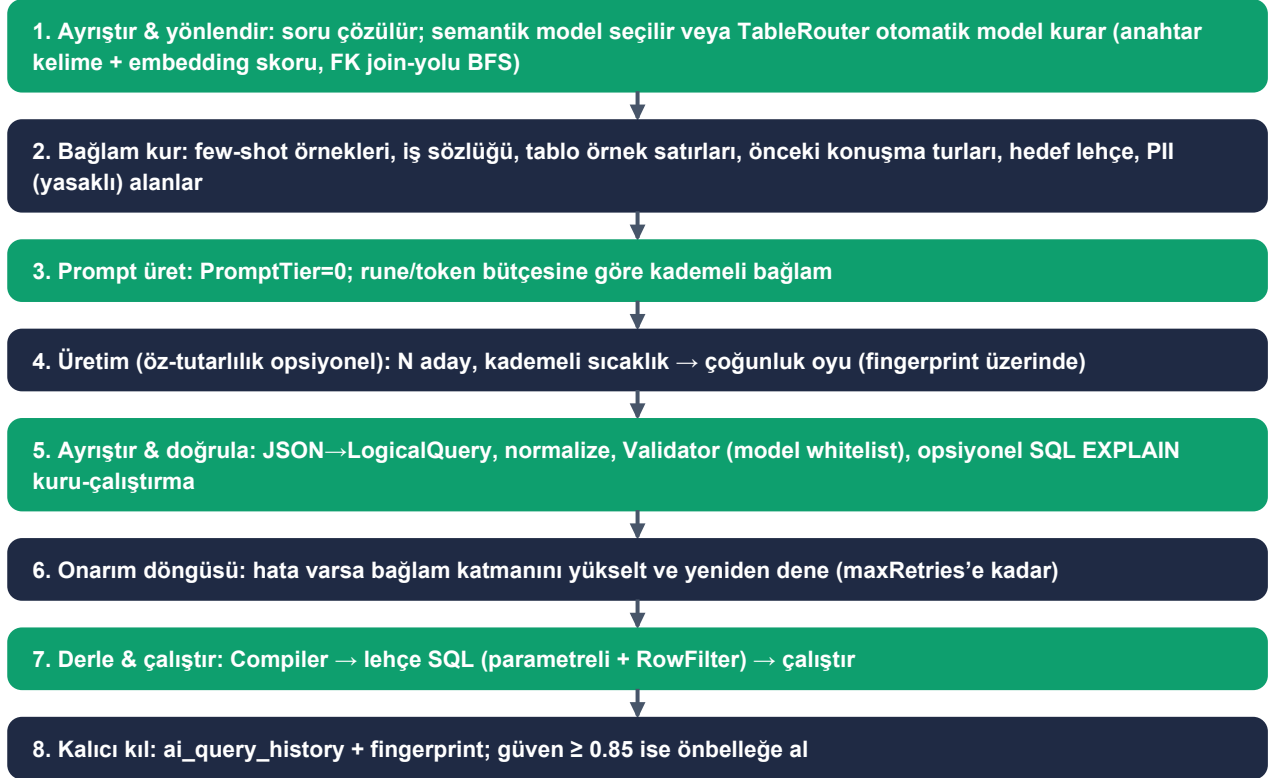
5.3 API Katmanı, Durum ve i18n

- **Fetch çekirdeği** (`apiClient.ts`): tipli `fetchJSON<T>`, `AbortController` zaman aşımı, otomatik `X-Locale` başlığı, Bearer token.
- **CSRF** (`csrf.ts`): tüm istekler `csrfFetch` üzerinden çift-gönderim deseniyle; güvenli metotlar token'ı yakalar, güvensiz metotlar `X-CSRF-Token` enjekte eder.
- **Durum & kimlik:** global store yok — Context + hook'lar (`AuthProvider`, `useApi`, `useAIJobs poller`); erişim token'ı yalnızca bellekte, refresh token `localStorage`'da, 14 dk'da bir sessiz yenileme.
- **i18n:** kütüphanesiz, **derleme-zamanı tip-güvenli** çeviri anahtarları; diller TR (varsayılan) ve EN; admin/auth sözlükleri tembel yüklenir.
- **Stil:** Vanilla CSS + BEM; `index.css` tasarım-sistemi çekirdeği (CSS değişkenleri, light/dark tema), özellik bazlı ~38 CSS dosyası.

6. Temel Akışlar (Pipeline'lar)

6.1 Text-to-SQL Akışı

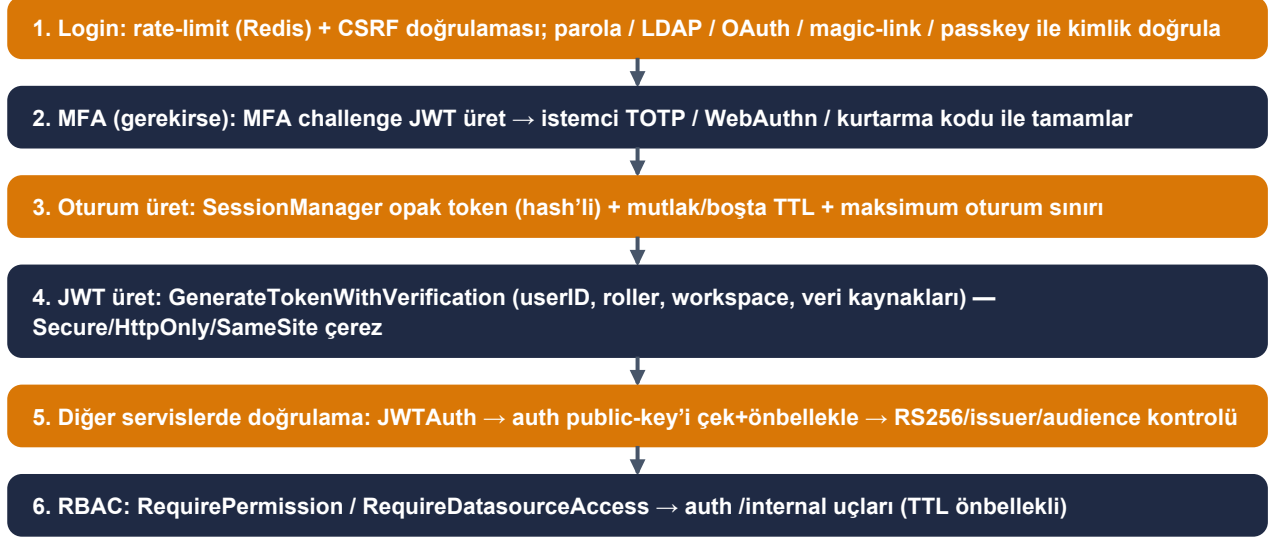
Platformun kalbi budur. Giriş: `POST /api/ai/query` (önizleme) veya `/api/ai/query/run`. Ağ geçidi `ai:query` iznini ve veri kaynağı erişimini uygular; sonra istek AI hattına girer. Aşağıdaki adımlar bir sıralı pipeline oluşturur:



Şekil 3 — Text-to-SQL kendi-kendini-onaran pipeline'ı. Güven skoru: $0.9 - 0.15 \cdot \text{doğrulama_hatası} - 0.10 \cdot \text{yeniden_deneme}$.

6.2 Kimlik Doğrulama (Auth) Akışı

Token modeli: **RS256 JWT erişim token'ları** (auth tarafından üretilir) + **opak, hash'lenmiş, DB tabanlı refresh oturumları**.



Şekil 4 — Kimlik doğrulama ve yetkilendirme akışı (RS256 JWT + opak DB oturumu + RBAC).

6.3 Semantik Katman ve Derleme Pipeline'ı

- **metric_graph.go**: kompozit modeller için metrik bağımlılık grafiği (MetricNode/Edge) kurulur; yayından/derlemeden önce DetectCircularDependencies ile döngü tespiti yapılır.
- **compiler.go**: LogicalQuery → CompiledQuery (SQL + args); join'ler join-grafiği üzerinden otomatik belirlenir; SELECT/WHERE/GROUP BY/HAVING/ORDER BY + pencere fonksiyonları kurulur. **Değerler asla string'e gömülmez (parametrelidir).**
- **expr_compiler.go**: ExprNode AST'i lehçe SQL'e çevrilir; PIIMaskingConfig ile PII maskeleye uygulanır.
- **validator.go**: alan/dimension/metric varlığı, tarih-filtre tipleri, satır üst sınırı kontrolleri; hatalar kararlı errmsg kodları taşır — AI onarım döngüsü bu kodları okur.
- **fingerprint.go**: sorgu + semantik model sürümü kanonikleştirilip kararlı hash üretilir; önbellek, geçmiş tekilleştirme ve öz-tutarlılık oylamasında kullanılır.

7. HTTP Katmanı ve Veri Katmanı

7.1 HTTP Middleware Yığını

Yığın (monolit Router): RequestID → trace-context yayılımı → istek logu → ReallP → chi Logger → Recoverer → SecurityHeaders (HSTS + CSP) → Locale → CORS (açık izin listesi). Auth servisi ek olarak CSRF ve RateLimiter ekler.

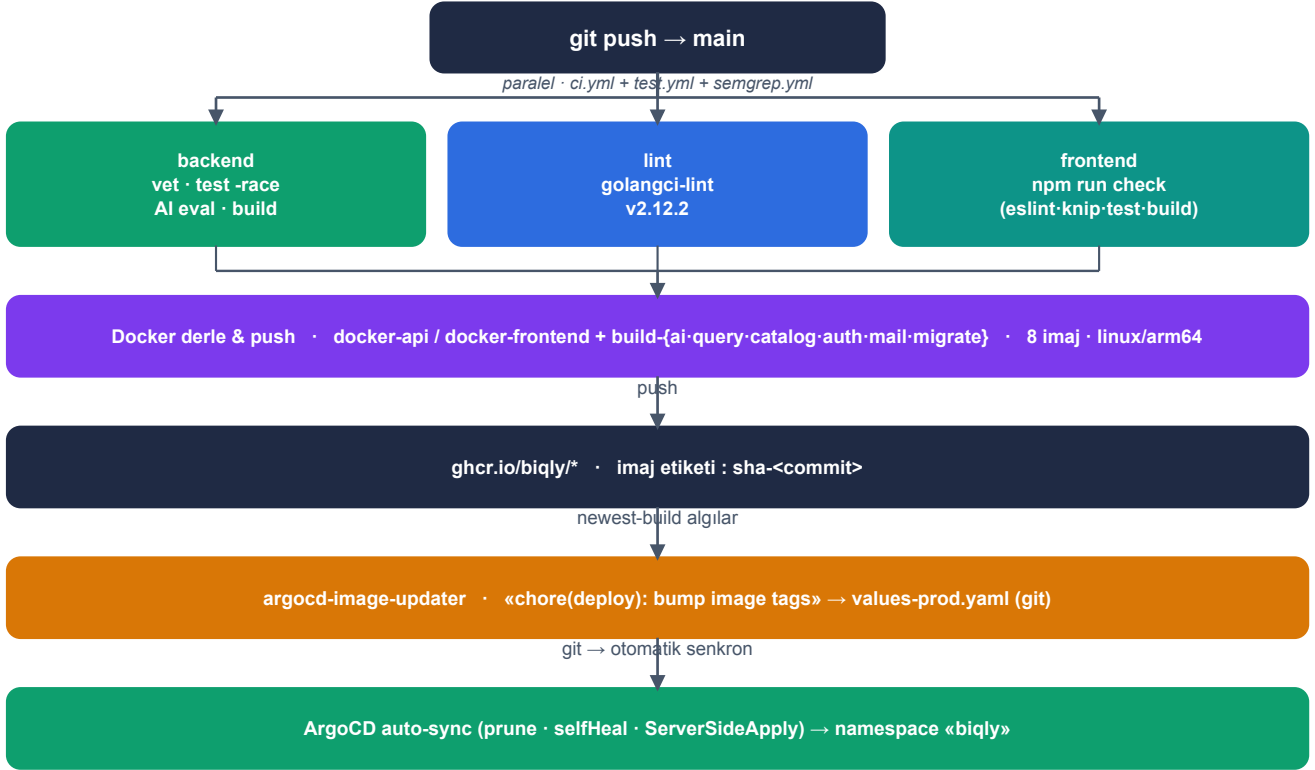
Rota grupları: `/health`, `/ready`, `/metrics` (Prometheus); JWT korumalı `/api/*` (catalog, query, ai grupları, her rotada izin + veri kaynağı erişim kontrolü); küme-içi `/internal/*` (X-Internal-Token + denetim). Bir `BI_*_SERVICE_URL` ayarlandığında ilgili `/api` grubu ters-proxy ile değiştirilir.

7.2 Veri Katmanı

Bileşen	Kullanım
PostgreSQL (pgx/v5)	Üç mantıksal DB: metadata, auth, mail. Harici kullanıcı veri kaynakları ayrıca MySQL/SQL Server/ClickHouse.
Göçler (dbmigrate)	Numaralı .sql dosyaları, schema_migrations + dirty-state tespiti; migrate/auth-migrate/mail-migrate binary'leri.
NATS / JetStream	Dayanıklı AI iş kuyruğu; yeniden teslim + dead-letter (maxDeliver). Uzun süren işler (batch describe, embedding).
Dragonfly / Redis	Rate-limit, AI yanıt önbelleği, RBAC veri-kaynağı erişim listeleri, oturum/OAuth durumu.
ai_jobs (metadata)	İş durumu/faz/ilerleme, çakışma tespiti, atomik TryMarkAIJobRunning (tek-sefer talep), bayat-iş toplama.

8. CI/CD Pipeline'ları ve Dağıtım

Monorepo; tek bir Kubernetes namespace'ine (biqly) Helm + ArgoCD GitOps ile dağıtılır. Konteyner kayıt defteri: `ghcr.io/biqly/*`. CI koşucu mimarisi ARM64'tür; imajlar `linux/arm64` olarak üretilir.



Şekil 5 — Commit'ten dağıtıma uçtan uca CI/CD ve GitOps pipeline'ı.

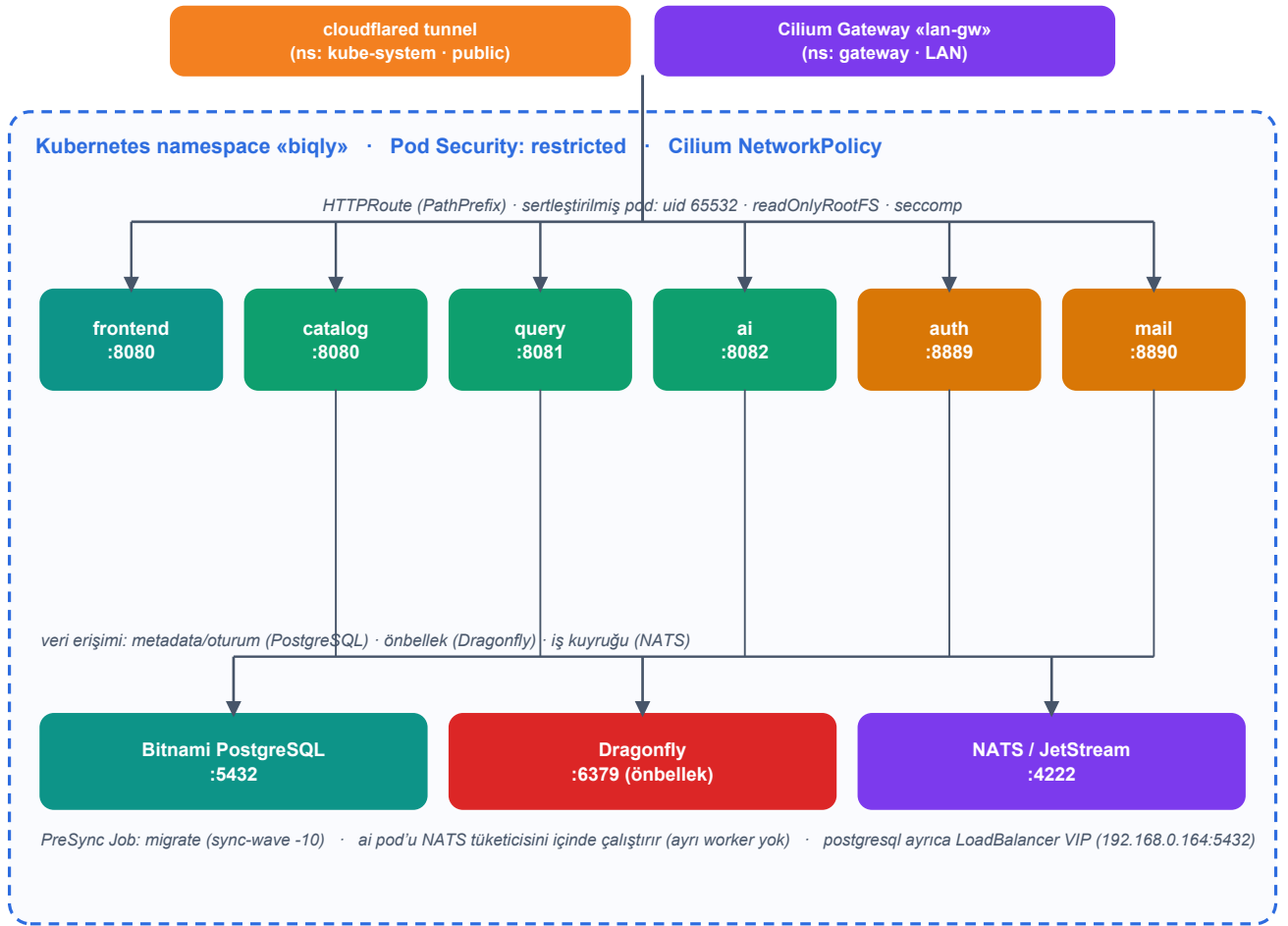
8.1 GitHub Actions İş Akışları

İş akışı	Amaç / Aşamalar
ci.yml	Ana hat. backend (vet → test -race+cov → AI eval regresyon kapısı → tüm cmd/* derle), lint (golangci v2.12.2), frontend (npm run check = eslint+prettier+knip+vitest+build), docker-api & docker-frontend push.
test.yml	Hafif kapı (PR'larda da): go test, golangci-lint, helm lint + template .
build-{ai,query,catalog,auth,mail,migrate}.yml	Servis başına Docker imaj derleme/push (QEMU+buildx, linux/arm64). Etiket: sha-<commit>. migrate her main push'ında derlenir.
semgrep.yml	SAST: p/security-audit, p/golang, p/react, p/typescript, p/owasp-top-ten → SARIF → fail-gate → GitHub Security. Push/PR + haftalık cron.

8.2 Docker İmajları

- **8 imaj**, hepsi çok-aşamalı, `linux/arm64`.
- **scratch tabanlı** (ai, query): yalnızca statik binary + CA paketi; `USER 65534`.
- **distroless tabanlı** (auth, mail): iki binary (servis + göç) + gömülü SQL göçleri; `nonroot`.
- **api**: `ARG SERVICE` ile çok-binary Dockerfile. **frontend**: Vite build → `nginx:alpine` (8080, SPA fallback + güvenlik başlıkları).

8.3 Kubernetes Runtime Topolojisi



Şekil 6 — «biqly» namespace'indeki çalışma-zamanı dağıtım topolojisi.

- **Helm umbrella chart:** namespace (Pod Security: restricted), Bitnami PostgreSQL, Dragonfly, NATS, Cilium NetworkPolicy'ler, gözlemlenebilirlik yığını (OTEL/Jaeger/Vector/Prometheus/Grafana/Alertmanager).
- **Sertleştirilmiş pod'lar:** runAsNonRoot, uid 65532, readOnlyRootFilesystem, tüm capability'ler düşürülür, seccomp RuntimeDefault, wait-for-postgres initContainer.
- **ArgoCD:** main dalından, values-prod.yaml ile otomatik senkron (prune + selfHeal); argocd-image-updater sha- etiketlerini git'e yazarak imajları otomatik yükseltir.
- **Göçler:** PreSync hook Job'ları (sync-wave ile sıralı); metadata göçü catalog imajının sha'sıyla kilit-adımda çalışır; ConfigMap migrations.biqly.io/revision anotasyonu senkron tetikler.
- **Edge / giriş:** public trafik kube-system'deki **cloudflared** named-tunnel'i (abi.ill.nl) üzerinden path-bazlı olarak doğrudan servislere; LAN içi yol Cilium «lan-gw» (gateway ns, LoadBalancer 192.168.0.179) + HTTPRoute'lar. Frontend pod'u nginx-unprivileged (dropped-caps init, ayrık API proxy timeout'ları).

9. Best-Practice Değerlendirmesi

Bu bölüm yedi mühendislik boyutunu kanıta dayalı olarak değerlendirir. Her boyut için güçlü yönler, riskler ve somut öneriler verilir. İşaretler:

• Güçlü • İzlenmeli • Risk

9.1 Güvenlik

Değerlendirme: ÇOK GÜÇLÜ (en güçlü boyut)

Güçlü yönler. Asimetrik **RS256 JWT** (issuer/audience doğrulamalı, MFA için ayrı audience); çift-gönderim CSRF + `subtle.ConstantTimeCompare` (zamanlama-güvenli); **mutlak + boşta** oturum süreleri, hash'li refresh token, cihaz parmak-izi; MFA/TOTP, WebAuthn, RBAC, LDAP, OAuth; sağlayıcı anahtarları **AES-256-GCM** ile şifreli. **Bu sürümde eklenenler:** tam güvenlik-başlığı seti — CSP, X-Frame-Options (DENY), X-Content-Type-Options, Referrer-Policy, COOP/CORP; **HSTS prod'da otomatik açık** (`BI_ENV=production`, fail-closed). Read-only SQL muhafızı sertleştirildi: allow/deny-list + çoklu-ifade engeli + **literal/yorum sıyırma**. **Bu turda:** prod-tespiti tek `env.IsProduction()` yardımcısında birleştirildi (api+auth ortak, testli); **catalog'daki kimliksiz-erişim açığı kapatıldı** (`/api/*` rotaları middleware'siz mount ediliyordu → artık JWT zorunlu); JWT admin-bypass zamanlama-güvenli ve boş-anahtar fail-closed; super-admin JWT kapısı; Cilium egress (catalog/query→auth) düzeltildi; CI'ya `govulncheck` + manuel semgrep tetikleyici eklendi.

Riskler / boşluklar. `BI_AUTH_ENABLED=false` iken savunma kısmen ağ-güvenine dayanır. Read-only koruması hâlâ tam SQL parser değil — fakat birincil savunma LogicalQuery whitelist doğrulaması ve parametrelili yürütmedir.

Öneriler. (1) Prod'da `BI_AUTH_ENABLED`'i zorunlu (fail-closed) bir invariant yapın. (2) `govulncheck/semgrep` taramalarını sürdürün.

9.2 Test

Değerlendirme: GÜÇLÜ — KAPSAM KAPISI EKLENDİ

Güçlü yönler. 200+ `_test.go` dosyası; yoğunluk tam da kritik yerlerde: handlers, ai, query, auth, semantic, security. Benchmark'lar; **race testi** zorunlu kapı; golden-file testleri; frontend vitest. **Bu sürümde eklenenler:** lehçe sürücüleri için testler (postgres/mysql/sqlserver/clickhouse her biri 2 dosya, dialect 4), config/dashboard/queue her biri 3 dosya; ve **zorunlu kapsam kapısı** — `scripts/coveragecheck` ile paket başına taban: dialect & sürücüler **%85**, config/dashboard **%80** (`test.yml`'deki coverage işi). Eval regresyon kapısı da sayısal eşiklerle CI'da (bkz. §9.6).

Riskler / boşluklar. `internal/queue` test aldı (3 dosya) ama kapsam-taban haritasında değil — test ediliyor fakat kapıya bağlı değil. Eval eşikleri determinist stub sağlayıcıyla 1.00; canlı-LLM doğruluk kayması ölçülüyor.

Öneriler. (1) `queue`'yu kapsam-taban haritasına ekleyin. (2) Periyodik (nightly) canlı-LLM eval koşusu ekleyerek gerçek doğruluk kaymasını izleyin.

9.3 Kod Kalitesi & Mimari

Değerlendirme: GÜÇLÜ — TEK KISMİ KALEM

Güçlü yönler. Temiz `cmd/internal/pkg` ayrımı; doğru kararlılık gradyanı. Hata yönetimi Go 1.26 ev-kurallarına güçlü uyum: **148× errors.Is, 11× errors.AsType, yalnızca 1 eski errors.As**. **Bu sürümde:** orkestratör `ProcessQuestion` ayrıştırıldı — `//nolint:gocyclo,gocognit,funlen` kaldırıldı, karmaşıklık skoru **12**'ye düştü; ~30 küçük yardımcı çıkarıldı (`generateWithRetries`, `parseAndValidate`, `buildSuccessResponse` vb.). Tüm ağaç `go build ./...` ve `go vet` □. **Bu turda ayrıca 4 yüksek-karmaşıklık fonksiyonu LOW/MEDIUM'a indirildi:** `Detector.Compare` 50→1, `datasourceDraft` 47→4, `SyncMetadata` 45→9, `Validator.Validate` 40→3 (toplam nolint 53→49); `pgarray` paketi `lib/pq` bağımlılığını tekleştirdi.

Riskler / boşluklar. `AIConfig` adlandırılmış alt-konfiglere geçirildi (`Query/Embedding/Translation/Router/Ambiguity` — gömülü→isimli), **ama tanrı-nesnesi skoru değişmedi** (21 alan / 13 metot / 93 dış çağrı = skor 60, hâlâ KRİTİK). Geri kalan çok-yüksek odaklar: `ValidateContext` (39), `ValidateComposite` (27), `PasswordPolicy.Validate` (25).

Öneriler. (1) `AIConfig`'in üst-seviye alanlarını gerçekten **dışarı taşıyın** (rename değil, taşıma). (2) Kalan 3 yüksek-karmaşıklık fonksiyonunu kademeli ayırıştırın.

9.4 Gözlemlenebilirlik

Değerlendirme: İYİ — TRACING ARTIK ENSTRÜMANTE

Güçlü yönler. Yapılandırılmış loglama (`log/slog`), **Prometheus** metrikleri, Helm'de tam yığın (OTEL collector, Jaeger, Vector, ServiceMonitor, Grafana, Alertmanager) ve **6 SLO-tarzı alarm kuralı**. **Bu sürümde önceki en büyük boşluk kapatıldı — OTEL dağıtık izleme uçtan uca kodda:** `otel` artık doğrudan bağımlılık (v1.44); servis giriş noktalarında `SetupTracing` ile OTLP-HTTP tracer provider (endpoint yoksa zarif no-op); router'da `otelhttp ingress span'i`; ve tam istenen **LLM→derle→çalıştır** yolunda adlandırılmış span'ler: `ai.ProcessQuestion`, `query.Compile`, `query.Execute`.

Riskler / boşluklar. Span kapsamı henüz sığ: veri-kaynağı sürücü çağrıları tek tek span'lenmiyor (yalnız executor sarmalıyor). Prometheus `Metrics struct'ı` (35 alan) gograph'ta HIGH işaretli.

Öneriler. (1) Sürücü/DB çağrıları ve few-shot/embedding gibi alt-fazlara span ekleyerek izleme derinliğini artırın. (2) Span'lere kritik öznitelikleri (model, attempt, fingerprint) ekleyin.

9.5 Performans

Değerlendirme: İYİ, ÖLÇÜME DAYALI

Güçlü yönler. Veri-kaynağı **bağlantı havuzu önbellesi** + `singleflight` tekilleştirme (thundering-herd önler, benchmark'lı); sürüm-farkında **sorgu fingerprint'i** (semantik model değişince önbellesi geçersiz kılar); AI yanıt + ambiguity önbellekleri; Dragonfly önbellek katmanı; CLAUDE.md'de ciddi sıcak-yol disiplini (`prealloc`, `strings.Builder`, `pprof`) ve golangci ile zorlanan `prealloc/perfsprint`; sıcak-yol JSON için `sonic/json-iterator`.

Riskler / boşluklar. **Bu sürümde:** Prometheus etiket kardinalitesi sınırlandı (853739e) ve gecikme artık **span'lerle atfedilebilir** (LLM vs derle vs çalıştır — §9.4). **LLM gecikmesi** hâlâ baskın maliyet; öz-tutarlılık oylaması (N aday) tur sayısını çoğaltır. Bu turda yeniden benchmark yapılmadı.

Öneriler. (1) Yeni span'lerden p99 gecikme dağılımını periyodik raporlayın. (2) Öz-tutarlılık N'inin konfig-kapılı ve makul prod varsayılanlı olduğundan emin olun.

9.6 AI / LLM Mühendisliği

Değerlendirme: OLGUN — AÇIK FARKLILAŞTIRICI

Güçlü yönler. İki aşamalı mimari: LLM ham SQL değil *LogicalQuery* üretir; semantik modele karşı doğrulanır ve *sonra* lehçe SQL'e derlenir — bu hem doğru tasarım hem de enjeksiyon savunmasının temelidir. **Onarım döngüsü** (yapısal hata bağlamıyla yeniden prompt + kademeli bağlam katmanları), **EXPLAIN dry-run** doğrulama kapısı, **öz-tutarlılık oylaması**, **güven skorlaması**, **belirsizlik (ambiguity) işleme** ve gerçek bir **eval koşum hattı** (golden case + LLM judge + regresyon testi) ile `cmd/export-sft` ince-ayar geri-besleme döngüsü. DB'den yönetilen, AES-GCM şifreli sağlayıcı/model. **Bu sürümde:** eval/regresyon paketi artık **CI kapısı** (`test.yml` + `ci.yml`) ve **açık sayısal eşiklerle** zorlanıyor (golden + benchmark mantıksal/yürütme oranı, `t.Fatal`); orkestratör `ProcessQuestion` ayrıştırıldı (§9.3).

Riskler / boşluklar. Eval eşikleri **determinist stub sağlayıcı** üzerinde 1.00 — yani harness/derleyici regresyonunu yakalar, canlı-LLM doğruluk kaymasını ölçmez.

Öneriler. (1) Periyodik (nightly) canlı-LLM eval koşusu ekleyerek gerçek doğruluk kaymasını izleyin. (2) Eval kapsamını yeni lehçe/edge senaryolarıyla genişletin.

9.7 DevX / Sürdürülebilirlik

Değerlendirme: MÜKEMMEL

Güçlü yönler. **golangci-lint v2** ~55 linter (`gosec`, `errorlint`, `contextcheck`, `bodyclose`, `rowserrcheck`, `sqlclosecheck`, `noctx`, `sloglint`, `fmt.Print*` ve `panic'i yasaklayan forbidigo`); frontend kapısı CI-eşdeğeri (ESLint + `jsx-a11y` + `security`, Prettier, **knip** ölü-kod tespiti, `vitest`, `build`); `deadcode -test` Go kapısında; zorunlu pre-commit denetimleri; gerçek i18n; 8 temiz `cmd/` giriş noktası; ADR dizini. **Bu sürümde:** repo kökü temizlendi — `*.test` ve `coverage.out` hem diskte hem git'te yok, `.gitignore` + `make clean` ile garanti; CI odaklı işlere bölündü (`test` / `eval` / `coverage` / `lint` / `helm`); yeni `make` hedefleri (`coverage-gate`, `eval-regression`).

Riskler / boşluklar. Kayda değer açık DevX riski kalmadı; küçük artık: bazı paketler kapsam-taban haritası dışında (örn. `queue`).

Öneriler. (1) Kapsam-taban haritasını kademeli genişletin. (2) ESLint uyarı tavanını zamanla 0'a doğru sıkın.

10. Sonuç ve Yol Haritası

Biqlly; canlı kümede **saf mikroservis** olarak çalışan (her servis ayrı Deployment; public edge **cloudflared** tüneli, LAN'da Cilium Gateway — her ikisi de path-bazlı yönlendirme), disiplinli, katmanlı-güvenlikli ve **standardın üzerindeki AI/text-to-SQL motoruyla** üretime hazır, geç-aşama bir kod tabanıdır. Bu doküman, önceki raporlardaki önerilerin uygulanmış halidir: ilk denetimdeki **8 boşluğun 7'si tamamen kapatıldı**; bu turda ayrıca 4 yüksek-karmaşıklık fonksiyonu indirildi, prod-tespiti birleştirildi ve catalog'daki kimliksiz-erişim açığı kapatıldı. Tüm değişiklikler kodda ve canlı kümede doğrulandı (`go build ./... · go vet · kubectl`).

Kalan öneriler (azalan öncelik)

Öncelik	Aksiyon	Etki
Orta	AIConfig'in üst-seviye alanlarını gerçekten dışarı taşı (rename değil)	Tek kalan KRİTİK tanrı-nesnesini kapatır
Orta	Kalan yüksek-karmaşıklık fonksiyonları (ValidateContext 39 · ValidateComposite 27 · PasswordPolicy.Validate 25)	Bakım borcu, test edilebilirlik
Orta	Periyodik (nightly) canlı-LLM eval koşusu	Stub-determinist eşiğin ötesinde gerçek doğruluk kayması
Orta	Prod'da BI_AUTH_ENABLED'ı zorunlu invariant yap	Ağ-güvenine düşmeyi engeller (fail-closed)
Düşük	İzleme derinliği (sürücü/DB span'leri); flaky TestMFABypassCodeFlow izolasyonu	Gözlem granülerliği / CI kararlılığı
Düşük	queue'yu kapsam-taban haritasına ekle	Hijyen / tutarlılık

*Bu doküman; kaynak kod statik analizine (gograph), doğrudan kaynak incelemesine, git geçmişi doğrulamasına ve **canlı Kubernetes kümesinin kubectl ile incelenmesine** dayanır. Skor kartı, geliştirme-sonrası kanıta dayalı niteliksel değerlendirmenin özetidir (ortalama ~4.4/5).*